Informe entrega 2

Sebastian Vargas
Jaime Alejandro Arevalo
Carlos Alberto Espinosa

2 Implementación del esquema de la base de datos:

Se crearon las nueve tablas del modelo: Estudiantes, Conductores, Pasajeros, Modelos, Vehiculos, Viajes, Lugares, Paradas y Solicitudes. Primero se crearon las tablas independientes y después las que hacen referencia a ellas, para poder agregar las llaves foráneas sin errores.
Los datos personales se almacenan en Estudiantes. Conductores y Pasajeros guardan únicamente el documento de identidad, que es su llave primaria y también una llave foránea hacia Estudiantes. Así se mantienen las relaciones propias de cada rol: los vehículos se asocian con los conductores y las solicitudes con los pasajeros, sin repetir los datos personales.
Se agregaron restricciones para evitar registros duplicados, campos obligatorios vacíos y referencias a registros inexistentes. También se restringieron los estados de viajes y solicitudes, el año de los vehículos y los valores de cupos, tarifa, distancia y duración.

3 Población de tablas:

Se cargaron datos ficticios en las nueve tablas, incluyendo 170 estudiantes: 20 conductores y 150 pasajeros. También se registraron 20 vehículos, 5 modelos, 10 lugares, 160 viajes, 320 paradas y 400 solicitudes.
Los datos se insertaron respetando las relaciones entre las tablas. Cada conductor tiene cinco viajes completados con tres pasajeros distintos en cada uno, para un total de 15 pasajeros diferentes. Además, se incluyeron viajes programados, en curso y cancelados, y solicitudes con diferentes estados para probar las funcionalidades de la aplicación.
Se verificaron las cantidades de conductores y pasajeros mediante consultas de conteo. Para comprobar los pasajeros distintos por conductor, se relacionaron Vehiculos, Viajes y Solicitudes, considerando únicamente solicitudes aceptadas de viajes completados.

4 Diseño de de sentencias SQL:

RFC1 – Demanda por localidad: Se utilizaron las tablas Solicitudes y Lugares, unidas por el atributo punto_recogida para identificar la localidad de cada solicitud. Se escogió INNER JOIN porque se necesitan las solicitudes que tienen un punto de recogida registrado en Lugares. La consulta filtra por fecha_solicitud y cuenta las solicitudes de cada localidad, incluyendo todos sus estados

SQL Commands

Language: SQL | Rows: 10 | Clear Command | Find Tables

```
SELECT L.localidad,
COUNT(*) AS cantidad_solicitudes
FROM Solicitudes S
INNER JOIN Lugares L
ON S.punto_recogida = L.punto_recogida
WHERE S.fecha_solicitud >= TO_DATE('01/09/2026', 'DD/MM/YYYY')
AND S.fecha_solicitud < TO_DATE('01/12/2026', 'DD/MM/YYYY')
GROUP BY L.localidad
ORDER BY L.localidad;
```

Results | Explain | Describe | Saved SQL | History

LOCALIDAD	CANTIDAD_SOLICITUDES
Chapinero	74
Kennedy	71
Suba	71
Teusaquillo	71
Usaquen	71

5 rows returned in 0.15 seconds Download

RFC2 – Ingresos estimados por localidad y período: Se utilizaron las tablas Solicitudes, Viajes y Lugares. Solicitudes se unió con Viajes con placa, fecha y hora_salida, ya que los tres atributos identifican un viaje. Luego se unió con Lugares con punto_recogida para obtener la localidad. Se usó INNER JOIN porque se necesitan las solicitudes asociadas a un viaje y a un lugar registrado. Se sumó la tarifa por cada solicitud aceptada, sacando los viajes cancelados y filtrando por la fecha del viaje

```
SELECT L.localidad,
SUM(V.tarifa) AS ingresos_estimados
FROM Solicitudes S
INNER JOIN Viajes V
ON S.placa = V.placa
AND S.fecha = V.fecha
AND S.hora_salida = V.hora_salida
INNER JOIN Lugares L
ON S.punto_recogida = L.punto_recogida
WHERE S.estado_solicitud = 'Aceptada'
AND V.estado IN ('Programado', 'En curso', 'Completado')
AND V.fecha >= TO_DATE('01/09/2026', 'DD/MM/YYYY')
AND V.fecha < TO_DATE('01/12/2026', 'DD/MM/YYYY')
GROUP BY L.localidad
ORDER BY L.localidad;
```

Results | Explain | Describe | Saved SQL | History

LOCALIDAD	INGRESOS_ESTIMADOS
Chapinero	544000
Kennedy	748000
Suba	442000
Teusaquillo	646000
Usaquen	340000

5 rows returned in 0.07 seconds Download

RFC3 – Viajes por conductor a través del tiempo: Se utilizaron las tablas Viajes, Vehiculos, Estudiantes, Paradas y Lugares. Viajes se unió con Vehiculos mediante placa, y Vehiculos con Estudiantes mediante documento_conductor y documento_identidad para obtener los datos del conductor. Se usó INNER JOIN para relacionar cada viaje con su

vehículo y conductor. Para filtrar por localidad, se relacionaron Paradas y Lugares con punto_recogida. La subconsulta se asoció con el viaje conn placa, fecha y hora_salida. Se utilizó EXISTS para comprobar que hubiera una parada en la localidad seleccionada sin contar varias veces el mismo viaje. Y también se filtraron los viajes completados por conductor y rango de fechas, y se agruparon por mes.

```sql
SELECT E.documento_identidad,
E.nombre,
E.apellidos,
TO_CHAR(V.fecha, 'YYYY-MM') AS mes,
COUNT(*) AS cantidad_viajes
FROM Viajes V
INNER JOIN Vehiculos Ve
ON V.placa = Ve.placa
INNER JOIN Estudiantes E
ON Ve.documento_conductor = E.documento_identidad
WHERE V.estado = 'Completado'
AND V.fecha >= TO_DATE('01/09/2026', 'DD/MM/YYYY')
AND V.fecha < TO_DATE('01/12/2026', 'DD/MM/YYYY')
AND E.documento_identidad = 1000000001
AND EXISTS (
SELECT *
FROM Paradas P
INNER JOIN Lugares L
ON P.punto_recogida = L.punto_recogida
WHERE P.placa = V.placa
AND P.fecha = V.fecha
AND P.hora_salida = V.hora_salida
AND L.localidad = 'Usaquen'
)
GROUP BY E.documento_identidad,
E.nombre,
E.apellidos,
TO_CHAR(V.fecha, 'YYYY-MM')
ORDER BY E.documento_identidad, mes;
```

Results | Explain | Describe | Saved SQL | History

DOCUMENTO_IDENTIDAD	NOMBRE	APELLIDOS	MES	CANTIDAD_VIAJES
1000000001	Ana	Gomez Lopez	2026-09	5

1 rows returned in 0.18 seconds Download